

Lab Report No. 03

Experiment Name

Implementation of DDA and Bresenham Line Drawing Algorithms with Execution Time Comparison and Bresenham Circle Drawing Algorithm Using OpenGL.

Objective

1. To draw two intersecting lines using DDA and Bresenham line drawing algorithms and compare their execution times.
 2. To draw an unfilled circle with center (250,250) and radius 10 using Bresenham's Circle Drawing Algorithm.
-

Theory/Introduction

Computer Graphics uses different algorithms to draw geometric shapes efficiently.

DDA (Digital Differential Analyzer) uses floating-point calculations to generate line points.

Bresenham Line Drawing Algorithm uses integer arithmetic, making it faster and more efficient.

Bresenham Circle Drawing Algorithm is used to draw circles by calculating only one-eighth of the circle and using symmetry to generate the remaining points.

In this experiment, two intersecting lines are drawn using DDA and Bresenham algorithms, and their execution times are compared. An unfilled circle is also drawn using Bresenham's Circle Drawing Algorithm.

Algorithm/Procedure

Task-01: Line Drawing and Time Comparison

1. Create a 600×600 OpenGL window.
2. Implement the DDA line drawing algorithm.
3. Implement the Bresenham line drawing algorithm.
4. Measure the execution time of both algorithms.
5. Draw the first line using DDA.
6. Wait for 3 seconds.
7. Draw the second intersecting line using Bresenham.
8. Display the execution times on the screen.

Task-02: Bresenham Circle Drawing

1. Set the circle center as (250,250).
 2. Set the radius as 10.
 3. Initialize the decision parameter.
 4. Plot symmetric points in all eight octants.
 5. Update the decision parameter.
 6. Continue until the circle is completed.
 7. Display the unfilled circle.
-

Source Code

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
import time

show_bresenham = False
dda_time = 0
bres_time = 0

def DDA(x1, y1, x2, y2):
    points = []

    start = time.perf_counter_ns()

    dx = x2 - x1
    dy = y2 - y1

    steps = max(abs(dx), abs(dy))

    x_inc = dx / steps
    y_inc = dy / steps

    x = x1
    y = y1

    for i in range(steps + 1):
        points.append((round(x), round(y)))
        x += x_inc
        y += y_inc

    end = time.perf_counter_ns()

    return points, end - start

def Bresenham(x0, y0, x1, y1):
    points = []
```

```

start = time.perf_counter_ns()

dx = x1 - x0
dy = y1 - y0

y = y0
p = 2 * dy - dx

for x in range(x0, x1 + 1):
    points.append((x, y))

    if p >= 0:
        y += 1
        p += 2 * dy - 2 * dx
    else:
        p += 2 * dy

end = time.perf_counter_ns()

return points, end - start

def drawCircle(cx, cy, r):
    glBegin(GL_POINTS)

    x = 0
    y = r
    p = 3 - 2 * r

    while x <= y:
        pts = [
            (cx+x, cy+y), (cx-x, cy+y),
            (cx+x, cy-y), (cx-x, cy-y),
            (cx+y, cy+x), (cx-y, cy+x),
            (cx+y, cy-x), (cx-y, cy-x)
        ]

        for px, py in pts:
            glVertex2i(px, py)

            if p < 0:
                p = p + 4*x + 6
            else:
                p = p + 4*(x-y) + 10
                y -= 1

            x += 1

    glEnd()

def draw_text(x, y, text):
    glRasterPos2f(x, y)

```

```

    for ch in text:
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, ord(ch))

def timer(value):
    global show_bresenham
    show_bresenham = True
    glutPostRedisplay()

def display():
    global dda_time, bres_time

    glClear(GL_COLOR_BUFFER_BIT)

    # DDA Line
    dda_points, dda_time = DDA(100,100,300,300)

    glColor3f(1,0,0)
    glBegin(GL_POINTS)
    for x,y in dda_points:
        glVertex2i(x,y)
    glEnd()

    draw_text(20,570,f"DDA Time: {dda_time} ns")

    # Bresenham Line after 3 seconds
    if show_bresenham:
        bres_points, bres_time = Bresenham(100,300,300,100)

        glColor3f(0,0,1)
        glBegin(GL_POINTS)
        for x,y in bres_points:
            glVertex2i(x,y)
        glEnd()

        draw_text(20,540,f"Bresenham Time: {bres_time} ns")

    # Circle
    glColor3f(0,1,0)
    drawCircle(250,250,10)

    glutSwapBuffers()

glutInit()
glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE)
glutInitWindowSize(600,600)

glutCreateWindow(b"Marajul Islam - 28")

glClearColor(1,1,1,1)

gluOrtho2D(0,600,0,600)

```

```
glutDisplayFunc(display)

glutTimerFunc(3000, timer, 0)

glutMainLoop()
```

Sample Input

Task-01

Line 1: (100,100) → (300,300)

Line 2: (100,300) → (300,100)

Task-02

Center: (250,250)

Radius: 10

Sample Output

- A 600×600 OpenGL window is displayed.
 - First line (DDA) appears immediately.
 - After 3 seconds, the second line (Bresenham) appears.
 - Execution times of both algorithms are shown.
 - An unfilled circle of radius 10 is drawn at (250,250).
-

Result

The DDA and Bresenham line drawing algorithms were successfully implemented and their execution times were displayed. An unfilled circle was also drawn successfully using Bresenham's Circle Drawing Algorithm.

Conclusion

The experiment was completed successfully. The performance of DDA and Bresenham algorithms was compared, and Bresenham's Circle Drawing Algorithm was used to draw an unfilled circle efficiently using integer arithmetic.